
Mazzon Daniele - Rubino Federico Giuseppe

Audio Processing Unit

Department of Information Engineering and Computer Science
University of Trento

Contents

1	SoC Main Architecture	1
1.1	Requirements	1
1.2	Interconnect and Memory Map	1
1.3	Instruction Memory and Boot Sequence	1
2	RV32-I Core	3
2.1	Microarchitecture	3
2.2	Datapath	3
2.3	Memory Interface and System Integration	4
3	APU Core	5
3.1	Main Architecture	5
3.2	Instruction Set Architecture	5
3.3	Control Unit	6
3.4	Parallel Memory Access	7
3.5	Audio Input/Output Unit	7
3.6	Fast Fourier Transform Unit	8
3.7	Vector Processing Unit	8
	3.7.1 Datapath	8
	3.7.2 Fixed-Point Arithmetic	9
4	Software Support	10
4.1	Drivers	10
4.2	Host Toolchain	10
4.3	Shader Toolchain	10

Chapter 1

SoC Main Architecture

The system is composed of two cores: a general-purpose RV32I CPU (Chapter 2), responsible for orchestration work, and a custom Audio Processing Unit (Chapter 3), an accelerator built for real-time digital audio processing. Figure 1.1 shows the top-level block diagram

Audio processing is regular, sample-rate-bound, and parallel, a poor fit for a general-purpose scalar core, so it is offloaded entirely to the APU, which processes audio in fixed-size grains and executes purpose-built shaders against a shared audio memory. The RV32I core handles everything that doesn't need to happen at sample rate: sequencing which APU program runs when, staging its parameters, and in general remain as a general purpose processing unit capable of MMIO. The CPU treats the APU purely as a memory-mapped peripheral, writing control registers and parameter tables and reading status, never touching the APU's internal audio memory itself.

1.1 Requirements

The system must process stereo audio in real time and apply multiple digital audio effects. Total system latency must remain below 15 ms, to avoid noticeable delay. To maintain acceptable fidelity, the sampling rate must be at least 48 kHz with a quantization of at least 16 bits per sample. It will be implemented in a Zedboard mounting an Zynq 7020 and an ADAU1761 audio chip, which contains a DAC and ADC circuits connected to the audio ports.

Given the constrained memory resources (630 kB, considering the Zynq 7020 140 BRAM blocks of 4,608 bytes each), the system uses the baseline quality settings: 16-bit audio at 48 kHz.

Because audio is processed in discrete blocks (grains), the grain size determines the latency. A grain size of 256 samples corresponds to approximately 5.33 ms of audio. In a pipeline with two buffers (one block period to fill the buffer and one to process it), the total latency is 10.67 ms. To meet real-time constraints across both channels, processing for a single channel's grain must complete within 2.66 ms.

1.2 Interconnect and Memory Map

The RV32I core is the sole bus master in the system. Its data-memory interface is bridged to AXI4-Lite and multiplexed through a SmartConnect interconnect to five memory-mapped slaves: a data memory, an instruction-memory load port (Section 1.3), the APU, and the UART and GPIO peripherals. Every peripheral lives in one flat address space, listed in Table 1.1.

1.3 Instruction Memory and Boot Sequence

Unlike its data memory, the RV32I core's instruction fetch is wired directly to a small multiplexer rather than through the AXI interconnect, keeping the fetch path off the shared bus entirely and free of any AXI handshaking latency. That multiplexer selects between two physically separate instruction memories based on one address bit: a small boot ROM, and a larger instruction memory holding the actual firmware. The boot ROM is a single-port memory whose contents are fixed at bitstream generation time; the firmware

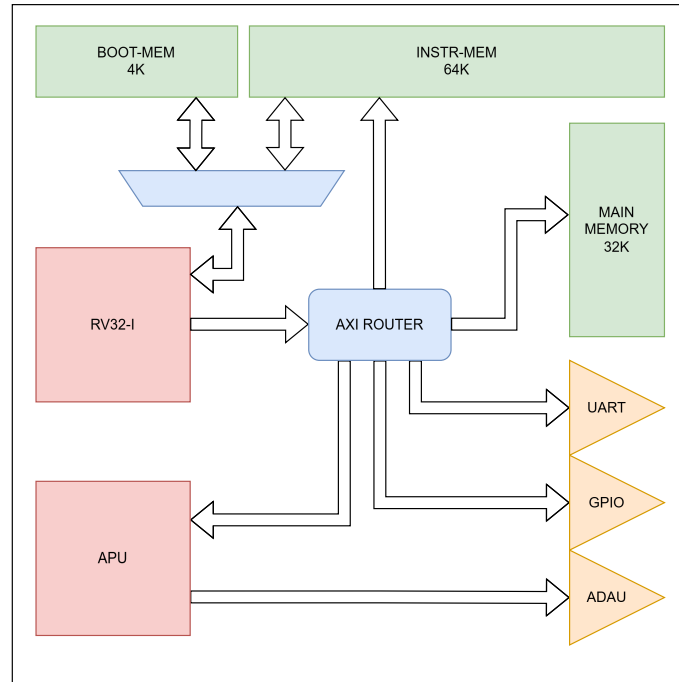


Figure 1.1: Overview of the SoC

Region	Base Address	Size	Destination
Data memory	0x00008000	32 KiB	<code>data_mem.bram</code>
Instruction memory (load)	0x00010000	64 KiB	<code>instr_mem.bram</code> (port B)
UART	0x00028000	128 B	<code>axi_uartlite_0</code>
GPIO	0x00029000	128 B	<code>axi_gpio_0</code>
APU	0x00030000	8 KiB	status/control, shader memory, parameter tables

Table 1.1: RV32I core address map.

memory is a true dual-port RAM: one port dedicated to the CPU’s direct fetch path, the other mapped onto the AXI bus precisely so the CPU can *write* its own instruction memory as ordinary data during bootloading.

On reset, the selector routes fetches to address 0 in the boot ROM, and the CPU begins executing the bootloader, a minimal program whose only job is to receive raw words over UART and write them to arbitrary addresses using store instructions the AXI data bus (Section 4). To load the real firmware, the bootloader is simply told to write it into the 0x00010000 region from Table 1.1, which physically lands in the second port of the same dual-port firmware memory the CPU fetches instructions from. Once the transfer is complete, the bootloader jumps to that address, the selector bit flips, and the CPU begins fetching from the newly-written firmware memory instead of the boot ROM.

Chapter 2

RV32-I Core

The control-plane processor is a multi-cycle implementation of the base RV32I integer instruction set. It is a strictly scalar, non-pipelined core.

2.1 Microarchitecture

The core is organized as a classic multi-cycle datapath: a small set of functional units (register file, ALU, branch comparator, load/store byte-lane logic, immediate sign-extender) share a single execution path, sequenced by a central finite-state-machine control unit. Every instruction, regardless of type, passes through 5 stages:

1. **FETCH** two cycles are spent fetching the instruction word from instruction memory, because of timing constraints of memory read latency.
2. **DECODE** the instruction's source register fields are driven onto the register file's read ports, and the resulting operands are latched into internal registers **A** and **B**.
3. **EXECUT** the ALU and branch unit operate on **A**, **B**, and the decoded immediate, producing **ALUOut** and the taken/not-taken decision **BranchOut**. **PC+4** is also computed and latched here.
4. **MEMORY** the program counter is committed (**ALUOut** for taken branches/jumps, otherwise **PC+4**). Load/store instructions assert the data memory enable here.
5. **WRITEBACK** for instructions that produce a register result, the appropriate value (ALU result, loaded data, link address, or immediate) is muxed onto the register file's write port.

A dedicated **stall** input freezes the entire FSM and every stage register in place, this is driven externally by the AXI bridge (Section 2.3) whenever a data-memory access is still outstanding on the system bus.

2.2 Datapath

The register file provides 32 general-purpose 32-bit registers with two asynchronous read ports and one synchronous write port. Writes to **x0** are discarded in hardware. The program counter is a single 32-bit register, updated once per instruction in the **MEMORY** state.

A single ALU implements all ten RV32I arithmetic/logic operations and is reused, via its **ADD** operation, for both branch/jump target computation and load/store effective-address calculation. A separate combinational branch unit evaluates the six RV32I branch conditions plus an always-taken mode shared by **JAL/JALR**. A dedicated load/store unit handles all sub-word memory access: it derives the correct byte-lane write mask for **SB/SH/SW** from the low two address bits, and performs the corresponding sign- or zero-extension for **LB/LH/LW/LBU/LHU** on the load path. Table 2.1 summarizes ISA coverage. The core implements the complete RV32I base integer instruction set for arithmetic, logic, control flow, and memory access.

Category	Instructions
R-type ALU	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
I-type ALU	ADDI, SLTI, SLTIU, XORI, ORI, ANDI, SLLI, SRLI, SRAI
Upper immediate	LUI, AUIPC
Control flow	JAL, JALR, BEQ, BNE, BLT, BGE, BLTU, BGEU
Loads	LB, LH, LW, LBU, LHU
Stores	SB, SH, SW

Table 2.1: RV32I instructions implemented by the core.

2.3 Memory Interface and System Integration

The core follows a Harvard organization, with entirely separate instruction and data memory buses. Instruction fetch is a simple synchronous read. The data memory interface is likewise a minimal synchronous protocol, an address, a read/write data pair, an enable, and a 4-bit per-byte write mask, with no built-in handshaking.

To connect this simple bus to the rest of the SoC over AXI4-Lite, a bridge module translates every data-memory access into a single AXI4-Lite transaction. The bridge asserts the CPU's `stall` input for the entire duration of an access and only releases it once the corresponding AXI response (`BVALID` for a write, `RVALID` for a read) is observed, guaranteeing at most one transaction at a time. This keeps the CPU's own control unit unaware of AXI's handshaking entirely.

Chapter 3

APU Core

3.1 Main Architecture

The Audio Processing Unit is built around that every kind of audio processing it can do — capturing audio, emitting audio, elementwise arithmetic on grains, and frequency-domain transforms — is expressed as a *Unit* that shares access to one common pool of audio memory. Rather than each Unit owning its own private buffers, all audio data lives in a single shared memory (referred then as A-RAM), and a Unit only ever becomes active when the APU’s control unit explicitly grants it access.

Physically, A-RAM consists of four dual-port block RAMs, addressed in row-major order: each row spans one 32-bit word from all four BRAMs simultaneously, and each 32-bit word packs two 16-bit audio samples, giving a max of 16 ordered audio samples at a time. A central arbiter, `aram_mux`, sits between the four physical BRAMs and every Unit’s memory ports; it reserves a fixed set of BRAM port connections for each Unit and multiplexes the four BRAMs’ real address/data buses onto whichever Unit is currently selected. Selection is single-active-unit: at any moment exactly one Unit (or none) drives the shared memory bus, chosen by a 3-bit `unit_select` signal driven by the control unit.

The control unit itself (Section 3.3) is a fetch-decode-execute state machine that reads and runs programs, “shaders”, from a dedicated shader instruction memory. Audio is processed in fixed-size *grains* of 256 samples: the Audio I/O Unit captures/emits one grain at a time from the audio codec, and every other Unit’s arithmetic operates on grain-sized (or multiple-of-grain-sized) spans of A-RAM. The RV32I core never talks to A-RAM or the Units directly. It interacts with the whole APU as one memory-mapped peripheral, a status/control register pair, the shader instruction memory, and two parameter tables, as described in Section 4.

3.2 Instruction Set Architecture

Operation	127-124	123	122	121-90	89-81	80-72	71-63	62-54	53-45	44-36	35-27	26-18	17-9	8-0
Audio in/out	opcode	-	-	-	-	-	-	-	-	-	buf start	buf len	op start	op len
Vector-vector	opcode	-	-	-	out buf start	out buf len	out op start	in buf start 2	in buf len 2	in op start 2	in buf start 1	in buf len 1	in op start 1	op len
Vector-scalar	opcode	-	-	-	out buf start	out buf len	out op start	-	-	scalar	in buf start	in buf len	in op start	op len
FFT/IFFT	opcode	fwd/inv	256/512 samples	-	out buf start	out buf len	out op start	-	-	-	in buf start	in buf len	in op start	op len

Table 3.1: Binary interface of APU instructions

Shader programs are a fixed-width, 128-bit instruction format (four 32-bit words), with the top 4 bits of every instruction reserved for the opcode regardless of which instruction it is. The instruction set covers: audio capture/emission (`AUDIO_IN`, `AUDIO_OUT`), vector arithmetic on the Vector Processing Unit (`ADD_VEC`/`ADD_SCALAR`, `SUB_VECTOR`/`SUB_SCALAR`, `MUL_VECTOR`/`MUL_SCALAR`, `MULC_VECTOR`), frequency-domain transforms (`FFT`, `IFFT`), a `LOAD` that use the param instruction memory to load audio samples from the outside of the APU, and `STOP`/`NOP`.

The single most important design decision in this ISA is that *no instruction field ever carries a literal value*. A field like a buffer’s start address, its length, or a scalar gain is not the value itself but it is an *offset* into a shared parameter table that the CPU has populated ahead of time via the APU driver (Section 4). This

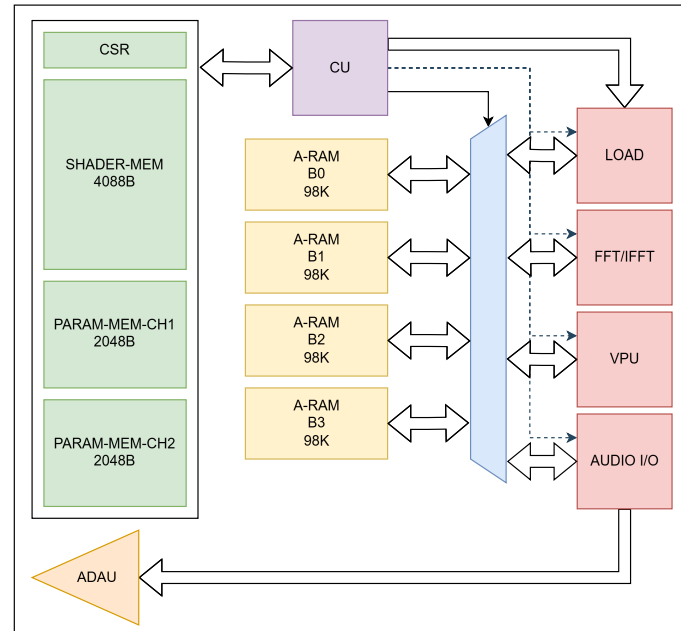


Figure 3.1: Overview of Audio Processing Unit architecture

has two consequences. First, the same fixed instruction encoding can address values with 32 bit precision without ever widening the instruction word, since the instruction only ever needs to be wide enough to hold an index, not the value itself. Second, and more deliberately, it is what makes stereo processing possible without duplicating shader code: there are two independent 512-word parameter tables, one per channel, and the exact same instruction stream is run twice, once per channel. A field like `buffer_start_reg=5` resolves against the left table while processing the left channel and the right table while processing the right channel, so one program produces two different results from two different sets of values the CPU staged at the same offsets with the same shader.

3.3 Control Unit

The APU's control unit is a six-state finite-state machine (IDLE, SETUP, FETCH, DECODE, LOAD, EXECUTE) that fetches shader instructions, resolves every parameter offset they reference, and dispatches exactly one Unit at a time to act on the result.

IDLE Polls a start bit the CPU writes into the APU's control register and the `new_grain` signal from the Audio IO Unit. When start is set high, it marks the APU as busy and moves to **SETUP**. When `new_grain` is set to 1, it updates the status register.

SETUP Computes the address of the first instruction word in shader memory.

FETCH Reads four consecutive 32-bit words from shader memory to assemble one complete 128-bit instruction.

DECODE Extracts the opcode from the fetched instruction. When processing the first channel, **STOP** cause a transition to **FETCH** and to the second channel. When processing the second channel it causes a transition to **IDLE**. Any other opcode proceeds to **LOAD**.

LOAD Loads the value of all parameters (for buffer identification and scalar values). Loops over each 9-bit address and loads it from the channel-specific section of parameter memory. It always loads 10 values regardless of the instruction.

EXECUTE Drives `unit_select` and the resolved buffer descriptors to whichever Unit the opcode targets, and waits for that Unit's own completion signal before returning to **FETCH** for the next instruction.

Since exactly one Unit is selected at a time during the **EXECUTE** state, Units never contend with each other for A-RAM: concurrency within the APU exists only inside a Unit, never between two Units at once.

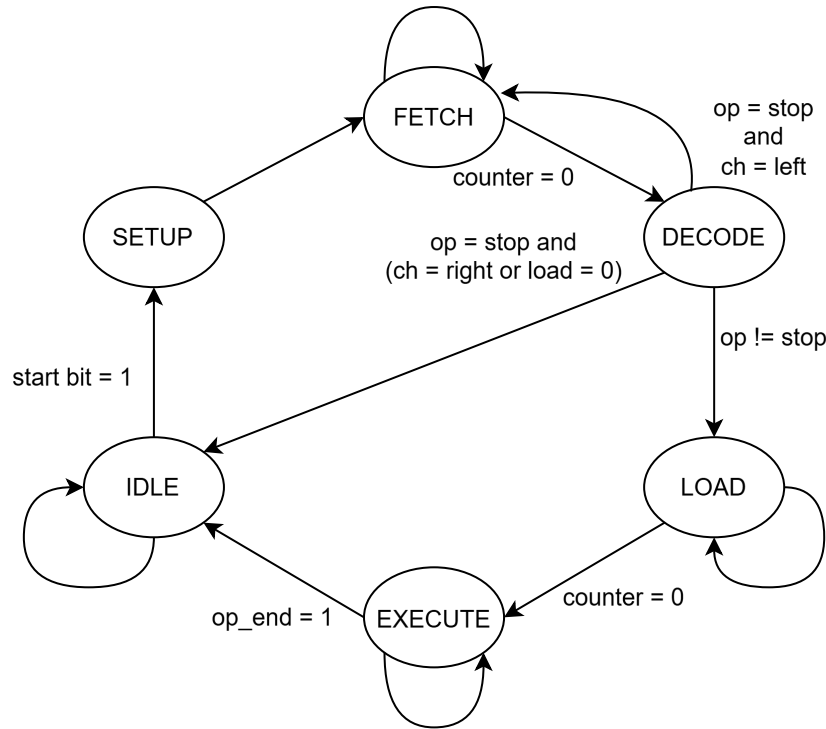


Figure 3.2: Finite state machine of the APU control unit

3.4 Parallel Memory Access

Every Unit that touches A-RAM does so through the same abstraction, referred to as a Buffer Management Unit (BMU): a sequencer that takes a start address, a length defining the span’s ring-buffer wraparound point, and the specific offset-and-length slice to actually access this run, and produces a stream of addresses (and, for writes, accepts a stream of data) without the Unit built on top of it needing to know anything about ring-buffer bookkeeping itself and parallel memory access. A BMU comes in a generic width, `LANES`, of 1, 4, or 8: at `LANES=1` it advances one sample per pulse (serial); at `LANES=4` or `LANES=8` it advances that x2 many samples per pulse, striped across the four physical BRAMs in parallel, trading physical port usage for throughput.

Physical access to A-RAM is gated by `aram_mux`: each Unit is permanently reserved a fixed set of BRAM port connections, sized to that Unit’s actual bandwidth needs. This keeps the memory-arbitration policy static and simple.

3.5 Audio Input/Output Unit

The Audio I/O Unit is decoupled from the rest of the APU’s instruction-driven timing: it runs continuously, independent of whether a shader is executing, interfacing with the on-board ADAU1761 audio codec (it’s build around an external IP that manages I2C ADAU configuration). As samples arrive from the codec are accumulated at 48KHz into a 256-sample buffer grain, that then is copied to A-RAM when asserted from the control unit. Symmetrically, when asserted from the control unit an output BMU reads a grain out of A-RAM and save in an internal grain buffer that then is serially streamed at 48KHz to the codec. Only once a full grain has been captured does the unit signal a new-grain-ready condition back to the control unit, which surfaces it in the APU’s status register, that is what firmware polls to know when it is worth triggering the next shader run.

Because capture and playback each need their own independent stream into and out of A-RAM, and both must keep pace with the codec’s real-time sample rate regardless of what the rest of the APU is doing, `AUDIO_IN` and `AUDIO_OUT` are exposed to the shader ISA as entirely separate opcodes, each with its own reserved bus into `aram_mux`, rather than being folded into a single "Audio I/O" opcode.

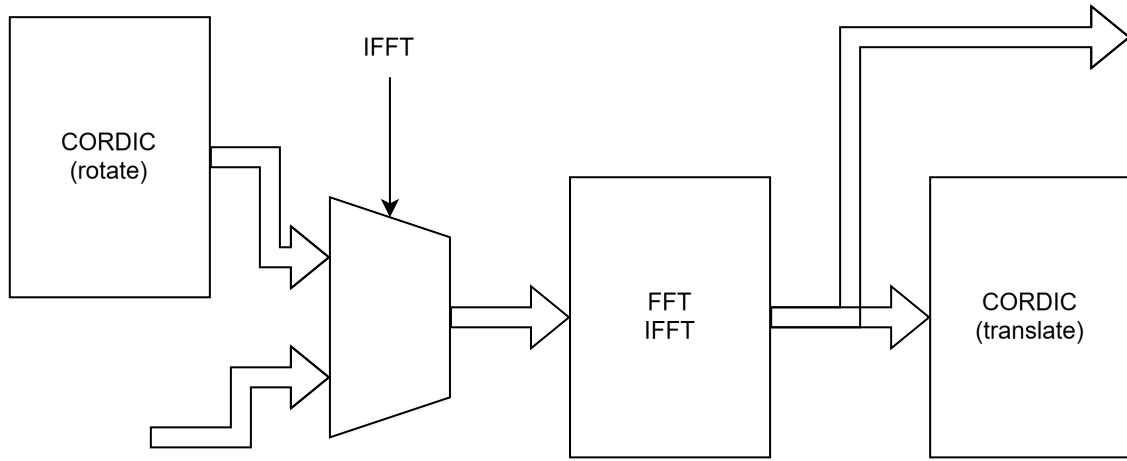


Figure 3.3: Design of the FFT Unit streaming pipeline

3.6 Fast Fourier Transform Unit

This FFT unit is a hardware accelerator designed for flexible fast Fourier transform. It operates as a dual-direction streaming engine, it seamlessly handles both Forward FFT and Inverse FFT operations on configurable 256-sample or 512-sample frame sizes.

To interface directly with memory, the architecture incorporates dedicated Read and Write Buffer Management Units. These units translate logical circular buffer parameters—such as base addresses and operation offsets—into physical memory accesses across four dual-port Block RAM (BRAM) banks using a single-lane, round-robin scheme.

The processing pipeline features domain conversions driven by Xilinx CORDIC IP cores: a Cartesian-to-Polar CORDIC core calculates magnitude and phase during Forward FFT, while a Polar-to-Cartesian CORDIC core reconstructs complex time-domain values during Inverse FFT. Data width alignment logic dynamically unpacks 32-bit memory words into 16-bit streaming samples for the FFT core, and packs dual 16-bit real outputs back into 32-bit words for storage. A four-state Finite State Machine (FSM) orchestrates configuration and pipeline execution of the blocks that communicate with the AXI4-Stream handshake protocol.

3.7 Vector Processing Unit

The Vector Processing Unit (VPU) implements elementwise arithmetic across A-RAM: addition, subtraction, and multiplication, each in a vector-vector form (two buffers combined elementwise) and a vector-scalar form (one buffer combined with a single broadcast value), plus a complex-multiply variant, which is useful for convolving audio grains in the frequency domain.

The VPU is pipelined (Fig 3.4): a lane-group of samples enters the pipeline, propagates through several stages (memory read, DSP execution, output realignment, memory write) so read, compute, and write for different lane-groups are processed simultaneously. Throughput is eight 16-bit samples each 2 cycle, processed by eight DSP48-based multiply-add cores instantiated in parallel. Three Buffer Management Units connect it to A-RAM: two independent read BMUs (LANES=4 each), matching the two operands a vector-vector instruction needs, and one write BMU (LANES=4) for the result.

3.7.1 Datapath

!!!!!!!!!!!!NEED REVIEW!!!!!!!!!!!!!!!!!!!!!!

The VPU processes eight 16-bit samples per cycle, one per lane, using eight DSP48-based multiply-add cores instantiated in parallel. Feeding them requires two independent input streams (the two operands of a vector-vector operation, or one operand plus a broadcast scalar) and one output stream, so the VPU instantiates three BMUs: two read BMUs (LANES=4 each) and one write BMU (LANES=4). `aram_mux` only reserves one eight-lane bus for the whole VPU (four BRAMs $\times$ two physical ports each), so the two read

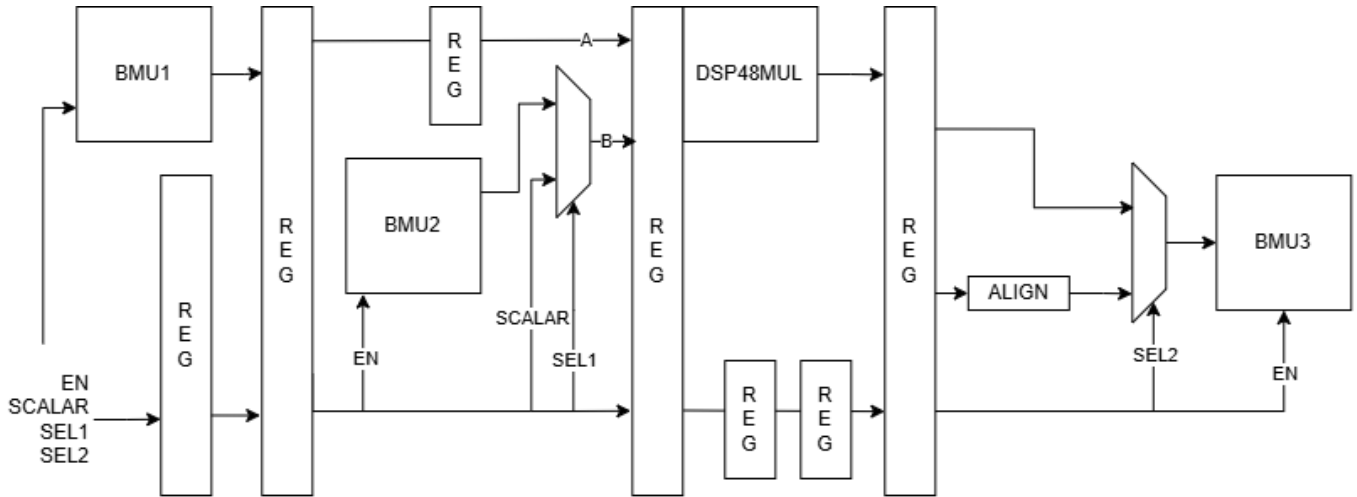


Figure 3.4: Vector Processing Unit pipeline architecture

BMUs share one physical port pair between them — read access uses one physical port of all four BRAMs, write access the other — and, since only one read BMU can be on the shared bus at a time, an internal time-multiplexing mux alternates which of the two read BMUs is actually driving it.

That time-multiplexing is what shapes the VPU's whole pipeline. A chain of staggered enable signals paces the two read BMUs' bus accesses so their pulses never land on the shared port in the same cycle, while a matching data pipeline carries each read BMU's fetched samples forward, in step, to the point where the DSP array consumes them; the DSP results are then packed back into 32-bit A-RAM words and handed to the write BMU. The nested control unit driving all of this (IDLE, SETUP, INJECT, HOLD, DRAIN, FINISH) starts both read BMUs together, then repeatedly injects and holds to keep the pipeline fed one lane-group at a time until the first read BMU reports done, at which point it drains the remaining pipeline stages into the write BMU and signals completion back to the main control unit.

3.7.2 Fixed-Point Arithmetic

Audio samples are carried as raw 16-bit signed integers throughout, but scalar operands (gains, mix levels) are represented in Q8.8 (FOR NOW) fixed-point. Multiplying a raw sample by a Q8.8 scalar produces a result scaled up by 2^8 , so the multiply-family operations (MUL_VECTOR, MUL_SCALAR, and the multiply half of MULC_VECTOR) apply an arithmetic (sign-extending) right shift by 8 bits to their DSP output before it is written back, realigning the result to the same Q0 (raw sample) scale as the inputs. Addition and subtraction need no such rescale, since both operands are already at the same scale, and are taken directly from the DSP's raw output.

Chapter 4

Software Support

The software stack is organized into three layers: bare-metal drivers executing on the RV32I core, host-side tooling used to load code onto the board, and an off-board toolchain responsible for assembling APU shader programs.

4.1 Drivers

Firmware interacts with each peripheral — UART, GPIO, and the APU — through a memory-mapped register `struct` accompanied by a small set of inline functions, following a uniform pattern throughout the driver layer. No interrupt mechanism is employed, all waiting is performed through polling loops, as the RV32I core provides no timing peripheral.

The APU driver (`apu.h`) constitutes the most significant component of this layer. It exposes the APU's status and control registers, its shader instruction memory, and its two (left- and right-channel) parameter tables through four operations: `apu_is_busy` and `apu_has_new_grain`, which poll status; `apu_load_shader`, which transfers an assembled shader's instruction words into shader memory; `apu_load_param`, which stages a value into both channel tables at a given offset; and `apu_start_shader`, which initiates execution and blocks until completion.

A further, smaller component is the bootloader, a minimal firmware image resident at the reset vector that implements the receiving end of the loading protocol. It accepts raw data words over UART, writes them to arbitrary memory addresses, and transfers control to the main firmware once instructed that the transfer is complete.

4.2 Host Toolchain

Two Python scripts, executed on a development PC, constitute the host-side tooling. `loader.py` implements the host side of the bootloader protocol: it transmits the instruction, data, and, optionally, shader images as address-tagged chunks over a serial connection, awaiting acknowledgment after each, before signaling completion and entering a live UART monitor. Destination addresses are read from `memory_map.ini`, ensuring the address map is maintained in a single location. `hex2coe.py`, by contrast, is employed exclusively at build time and is unrelated to the runtime loading path: it converts a `.hex` word list into a Vivado `.coe` file used to pre-load a BRAM's initial contents at synthesis time.

4.3 Shader Toolchain

The generation of APU shaders is handled entirely off-board by a dedicated Python toolchain.

The instruction encoding is not embedded within the toolchain's implementation but is instead specified declaratively in `isa.yaml`, which defines the instruction width, the position of the opcode field, and, for each opcode, a name and an ordered list of fields expressed as bit ranges.

The assembler (`assembler.py`) translates a `.shader` source file into a `.hex` word stream in two principal stages. A preprocessing stage resolves `.define`, `.param`, and `.macro` directives, together with a block syntax, into a flat sequence of plain instruction lines. An encoding stage subsequently resolves each mnemonic against the table derived from `isa.yaml` and packs its fields into a 128-bit instruction, which is emitted as four 32-bit words. Two auxiliary outputs are provided to support development: `--emit-flat`, which produces the macro-expanded instruction listing prior to encoding, and `--emit-params`, which generates a C header defining one symbolic constant per `.param`, allowing firmware to reference, for instance, `GAIN` rather than an unlabeled offset. A companion utility, `disassemble.py`, performs the inverse transformation, decoding a `.hex` file into mnemonic form; this was employed throughout development to verify that a given shader was encoded as intended.